Rather than using `std::async()`, you could write your own `spawn_task()` function as a simple wrapper around `std::packaged_task` and `std::thread`, as shown in listing 4.14; you'd create a `std::packaged_task` for the result of the function call, get the future from it, run it on a thread, and return the future. This wouldn't itself offer much advantage (and indeed would likely lead to massive oversubcription), but it would pave the way to migrate to a more sophisticated implementation that adds the task to a queue to be run by a pool of worker threads. We'll look at thread pools in chapter 9. It's probably worth going this way in preference to using `std::async` only if you really know what you're doing and want complete control over the way the thread pool is built and executes tasks.

Anyway, back to `parallel_quick_sort`. Because you just used direct recursion to get `new_higher`, you can just splice it into place as before ❸. But `new_lower` is now a `std::future<std::list<T>>` rather than just a list, so you need to call `get()` to retrieve the value before you can call `splice()` ❹. This then waits for the background task to complete and *moves* the result into the `splice()` call; `get()` returns an rvalue reference to the contained result, so it can be moved out (see appendix A, section A.1.1 for more on rvalue references and move semantics).

Even assuming that `std::async()` makes optimal use of the available hardware concurrency, this still isn't an ideal parallel implementation of Quicksort. For one thing, `std::partition` does a lot of the work, and that's still a sequential call, but it's good enough for now. If you're interested in the fastest possible parallel implementation, check the academic literature.

Listing 4.14. A sample implementation of `spawn_task`

```
template<typename F,typename A>
std::future<std::result_of<F(A&&)>::type>
    spawn_task(F&& f,A&& a)
{
    typedef std::result_of<F(A&&)>::type result_type;
    std::packaged_task<result_type(A&&)>
        task(std::move(f)));
    std::future<result_type> res(task.get_future());
    std::thread t(std::move(task),std::move(a));
    t.detach();
    return res;
}
```

Functional programming isn't the only concurrent programming paradigm that eschews shared mutable data; another paradigm is CSP (Communicating Sequential Processes),[3] where threads are conceptually entirely separate, with no shared data but with communication channels that allow messages to be passed between them. This is the paradigm adopted by the programming language Erlang (http://www.erlang.org/) and by the MPI (Message Passing Interface) (http://www.mpi-forum.org/) environment commonly used for high-performance computing in C and C++. I'm sure that by now you'll be unsurprised to learn that this can also be supported in C++ with a bit of discipline; the following section discusses one way to achieve this.

[3] *Communicating Sequential Processes*, C.A.R. Hoare, Prentice Hall, 1985. Available free online at http://www.usingcsp.com/cspbook.pdf.

4.4.2. Synchronizing operations with message passing